PR

Supervised

Unsupervised

Number of clusters known

Number of clusters unknown

Partitional (outputs single clustering)

Hierarchical (All possible clustering of given dataset)

Measurement Space → Feature Space → Decision Space

A Typical PR system

Decision function : $X = \begin{pmatrix} F \\ L \end{pmatrix}$

$D(X) > 0$ for class 1

$D(X) \leq 0$ for class 2

$D(X) = 0$ decide arbitrarily

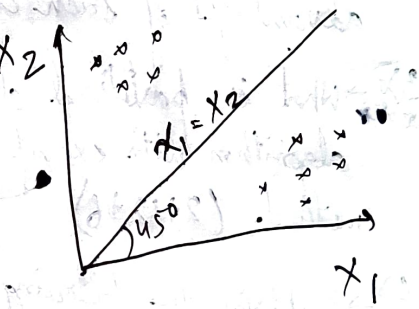
Equation & coefficient

$w_1 x_1 + w_2 x_2 + w_3 \geq 0$

$w_1 = 1$

$w_2 = -1$

$w_3 = 0$



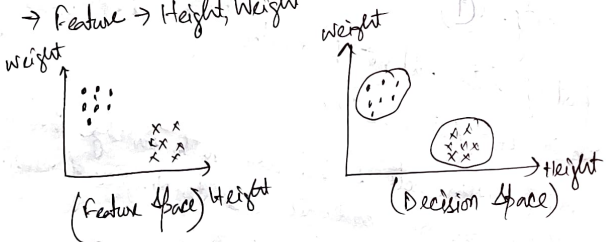
A 29) A naive Bayes classifier always assumes that each feature x_i is conditionally dependent of every other feature x_j for $j \neq 1$ (F)

Example: Given a set of professional basketball players and wrestlers, how to recognize an individual of the set to be a player of any one of these two games

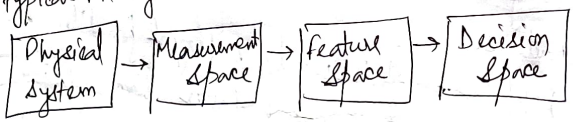
→ Measurements: Age, Qualification, height weight

→ Basis of feature selection: Basketball players are taller and slimmer, where as wrestlers are comparatively shorter and heavier.

→ Feature → Height, Weight



Steps in typical PR system →



Pattern:-

A pattern is a description of an object that we are going to recognize. Description means a set of measurements, but all these measurements are not significant in the context of recognition.

⊙ When each feature is considered as a component of a vector, it's called a pattern vector or feature vector.

A pattern vector is represented as a point in n-dimensional euclidian space → feature space

⊙ When different regions of the feature space are identified to belong to different classes, then such a space is called a decision space.

Pattern classification by Decision functions:-

Let $C_1, C_2, \ldots C_j, \ldots C_m$ be designated as the m possible pattern classes in an N-dimensional feature vector space S_X and let $X = [x_1, x_2, \ldots x_n, \ldots x_N]$ be the unknown pattern vector where x_n represents the nth feature measurement

→ If the pattern X is a member of class C_k, the decision function $D_k(X)$ associated with the class C_k $[k = 1, 2, \ldots m]$ must then possess the largest value.

Decide $X \in C_k$ if $D_k(X) > D_j(X)$ with $k \neq j$, $[k, j = 1, 2, \ldots m]$

→ Ties are resolved arbitrarily. The decision boundary is -

$$D_k(X) - D_j(X) = 0 \quad \text{where } k \neq j,\ k, j = 1, 2, \ldots m$$

The success depends on two factors — $d(x) = w_1 x_1 + w_2 x_2 + w_3$

(a) The form of $d(x)$

(b) One's ability to determine its coefficients.

General n-dimensional form of decision function :-

A general linear decision function is of the form —

$$d(X) = w_1 x_1 + w_2 x_2 + \dots w_n x_n + w_{n+1}$$

$$= W_0' X + w_{n+1} \quad \text{where } [W_0 = (w_1, w_2, \dots w_n)']$$

W_0 : Weight/Parameter vector

$$d(X) = W'X, \text{ where } X = (x_1, x_2 \dots x_n, 1)$$

$$W = (w_1, w_2, \dots w_n, w_{n+1})$$

It is augmented pattern and weight vectors

In the two-class case, a decision function $d(X)$ is assumed to have the property,

$$d(X) = W'X \begin{cases} > 0 \text{ if } X \in \omega_1 \\ < 0 \text{ if } X \in \omega_2 \end{cases}$$

for multiclass scenarios —

Case-1 : Each pattern class is separable from the other classes by a single decision surface. In this case there are M decision functions with the property.

$$d_i(X) = W_i' X = \begin{cases} > 0 \text{ if } X \in \omega_i \\ < 0 \text{ otherwise} \end{cases}, \quad i = 1, 2, \dots M$$

where $W_i = (w_{i1}, w_{i2}, \dots w_{in}, w_{i,n+1})$ is the weight vector.

Case-2 : Each pattern class is separable from every other individual class by a distinct decision surface, that is the classes are pairwise separable. In this case there are $M(M-1)/2$ decision surface. The decision functions are of the form $d_{ij}(X) = W_{ij}' X$

and have the property that if
x belongs to class ω_i then $d_{ij}(x) > 0$ for all $j \neq i$
These functions also have the property that

$$d_{ij}(x) = -d_{ji}(x)$$

Geometric Properties of linear decision functions (Hyperplane properties)

The equation of the surface separating the pattern classes is obtained by letting the decision functions be equal to 0,

$$d(x) = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + w_{n+1} = 0 \quad \text{---} (1)$$

In case 1,

$$d_i(x) = w_{i1} x_1 + w_{i2} x_2 + \dots + w_{in} x_n + w_{i,n+1} = 0 \quad \text{---} (2)$$

In Case 2,

$$d_{ij}(x) = w_{ij1} x_1 + \dots + w_{ijn} x_n + w_{ij,n+1} > 0 \quad \text{---}$$

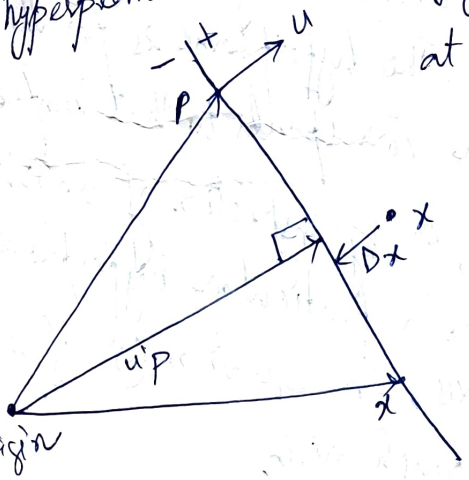
In General

$$d(x) = W_0' X + w_{n+1} = 0 \quad \text{---} (3)$$

where $W_0 = (w_1, w_2, \dots w_n)'$

Equation (3) is recognized as the equation of a line when $n=2$ as the equation of a plane when $n=3$. when $n>3$, it's the equation of hyperplane

A hyperplane is schematically —



Let u be a unit normal to the hyperplane at some point p and oriented to the positive side of the hyperplane.

Now, $u'(x-p) > 0 \quad \text{---} (4)$

or, $u'x = u'p \quad \text{---} (5)$

Dividing eq (3) by $\|W_0\| = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$

$$\frac{W_0' X}{\|W_0\|} = -\frac{W_{n+1}}{\|W_0\|} \quad \text{---} (6)$$

Comparing equation (5) and (6) we see that the unit normal to the hyperplane is given by —

$$u = \frac{W_0}{\|W_0\|} \quad \text{---} (7)$$

also, $$u'p = -\frac{W_{n+1}}{\|W_0\|} \quad \text{---} (8)$$

(i) It is seen, by comparing equation 8 with the fig that the absolute value of $u'p$ represents the normal distance from the origin to the hyperplane.

so, $$D_u = \frac{|W_{n+1}|}{\|W_0\|} \quad \text{---} (9)$$

(ii) The normal distance D_x from the hyperplane to an arbitrary point x is given by

$$D_x = |u'x - u'p| = \left| \frac{W_0' X + W_{n+1}}{\|W_0\|} \right|$$

(iii) The unit normal u indicates the orientation of the hyperplane. If any component of u is 0, the hyperplane is parallel to the coordinate axis which corresponds to that component. Therefore, since, $u = \frac{W_0}{\|W_0\|}$, so by inspection of vector W_0 it is possible to tell wheather a particular hyperplane is parallel to any of the coordinate axes. In equation (9) if $W_{n+1} = 0$ the hyperplane passes through origin.

Generalized Decision functions :-

$$d(X) = w_1 f_1(X) + w_2 f_2(X) + \cdots w_k f_k(X) + w_{k+1}$$

$$= \sum_{i=1}^{k+1} w_i f_i(X) \quad \cdots \quad (1)$$

where $\{f_i(X)\}[i = 1, 2, \cdots k]$ are real, single-valued functions of the pattern X, $f_{k+1}(X) = 1$, and $k+1$ is the no. of terms used in the expansions.

It represents infinite variety of decision functions depending on the choice of functions $\{f_i(X)\}$ and the no. of terms.

Now, $$X^* = \begin{bmatrix} f_1(X) \\ f_2(X) \\ \vdots \\ f_k(X) \\ 1 \end{bmatrix} \quad \cdots \quad (2)$$

Using Eq-2 we may express (1) as $d(X) = W' X^* \quad \cdots \quad (3)$

X^* is simply a k-dimensional vector which has been augmented by 1. Eq (3) represents a linear function with respect to the new patterns X^*.

Pattern Classification by Distance functions:-

The proximity of an unknown pattern to the patterns of a class will serve as a measure for its classification, so the method is called minimum-distance pattern classification. And the system which does this is called minimum-distance classifier.

Single Prototypes

In this case, the patterns of each class tend to cluster tightly about a typical or representative pattern for that class.

Consider M pattern classes and assume that these classes are representable by prototype patterns, $z_1, z_2, \ldots z_M$

The euclidean distance between an arbitrary pattern vector x and the ith prototype $-$ $D_i = \|x - z_i\| = \sqrt{(x-z_i)'(x-z_i)}$ ... (1)

x is assigned to class ω_i if $D_i < D_j$ $\forall$ $i \neq j$, Ties are resolved arbitrarily.

$$D_i^2 = \|x - z_i\|^2 = (x-z_i)'(x-z_i)$$
$$= x'x - 2x'z_i + z_i'z_i$$
$$= x'x - 2\left(x'z_i - \frac{1}{2}z_i'z_i\right)$$

$x'x$ is independent of i, choosing the minimum D_i^2 is equivalent to choosing maximum of $\left(x'z_i - \frac{1}{2}z_i'z_i\right)$

$$d_i(x) = x'z_i - \frac{1}{2}z_i'z_i \qquad [i = 1, 2, \ldots M]$$

where x is assigned to class ω_i if $d_i(x) > d_j(x)$ $\forall$ $j \neq i$

If z_{ij} $[j = 1, 2, \ldots n]$ are the components of z_i and we let $w_{ij} = z_{ij}$, $w_{i,n+1} = -\frac{1}{2}z_i'z_i$ and $x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \\ 1 \end{bmatrix}$

then, $d_i(x) = w_i'x$

Pattern Classification by Distance functions:-

The proximity of an unknown pattern to the patterns of a class will serve as a measure for its classification, so the method is called minimum-distance pattern classification. And the system which does this is called minimum-distance classifiers.

Single Prototypes

In this case, the patterns of each class tend to cluster tightly about a typical or representative pattern for that class.

Consider M pattern classes and assume that these classes are representable by prototype patterns, $z_1, z_2, \ldots z_M$

The euclidean distance between an arbitrary pattern vector x and the ith prototype — $D_i = \|x' - z_i\| = \sqrt{(x-z_i)'(x-z_i)}$ ... (1)

x is assigned to class ω_i if $D_i < D_j \ \forall\ i \neq j$, Ties are resolved arbitrarily.

$$D_i^2 = \|x - z_i\|^2 = (x-z_i)'(x-z_i)$$
$$= x'x - 2x'z_i + z_i'z_i$$
$$= x'x - 2\left(x'z_i - \frac{1}{2}z_i'z_i\right)$$

$x'x$ is independent of i, choosing the minimum D_i^2 is equivalent to choosing maximum of $\left(x'z_i - \frac{1}{2}z_i'z_i\right)$

$$d_i(x) = x'z_i - \frac{1}{2}z_i'z_i \qquad [i = 1, 2, \ldots M]$$

where x is assigned to class ω_i if $d_i(x) > d_j(x)$ $\forall\ j \neq i$

If z_{ij} $[j = 1, 2, \ldots n]$ are the components of z_i and we let $w_{ij} = z_{ij}$, $w_{i,n+1} = -\frac{1}{2}z_i'z_i$ and $X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \\ 1 \end{bmatrix}$

then, $d_i(x) = w_i'X$

where $W_i = (w_{i1}, w_{i2}, \ldots w_{i,n+1})'$

⊙ The linear decision surface separating every pair of prototype points z_i and z_j is the hyperplane which is the perpendicular bisector of the line-segment joining the two points. Thus, minimum-distance classifiers are a special case of linear classifiers.

Multi Prototypes :-

Each class is characterized by several prototypes, that is each pattern of class w_i tends to cluster about one of the prototypes $z_{i1}, z_{i2}, \ldots z_{iN_i}$ where N_i is the no. of prototypes in the ith pattern class.

Let, the distance function between an arbitrary pattern x and class w_i be denoted by —

$$D_i = \min \left\| X - z_i^l \right\| , \quad l = 1, 2, \ldots, N_i$$

$$d_i(X) = \max \left\{ (x' z_i^l) - \frac{1}{2} (z_i^l)' z_i^l \right\} \quad [l = 1, 2, \ldots N_i]$$

x is placed in class w_i if $d_i(X) > d_j(X) \ \forall \ j \neq i$

Measure of similarity :-

To define a data cluster, it is necessary to first define a measure of similarity which will establish a rule for assigning patterns to the domain of a particular cluster center, Euclidean distance, $D = \| X - Z \|$

Mahalnobis Distance form x to m

$$D = (X - m)' C - (X - m)$$

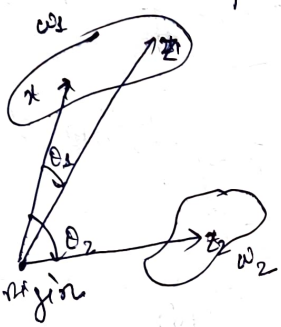
It is a useful measure of similarity when statistical properties are being explicitely considered. C is the covariance matrix of a pattern population, m is mean vector and x is a variable pattern.

Non-metric Similarity function:

$$S(x,z) = \frac{x'z}{\|x\|\,\|z\|}$$

which is the cosine of the angle between the vectors x and z, is maximum when x and z are oriented in the same direction with respect to the origin.

This measure of similarity is useful when cluster regions tend to develop along principal axes.



$$S(x,z_1) = \cos\theta_1 = \frac{x'z_1}{\|x\|\,\|z_1\|}$$

$$S(x,z_2) = \cos\theta_2 = \frac{x'z_2}{\|x\|\,\|z_2\|}$$

The use of this similarity measure is governed by certain qualifications such as sufficient separation of cluster regions and with respect to each other as well as with respect to the coordinate system origin.

When the patterns under consideration are binary valued with 0, 1 elements, the similarity function may be given an interesting non-geometrical interpretation.

$x'z \Rightarrow$ # of attributes shared by x and z.

$\|x\|\|z\| = \sqrt{(x'x)(z'z)} \Rightarrow$ geometric mean of the no. of attributes possessed by x and by z.

$S(x,z) \Rightarrow$ measure of common attributes possessed by the binary vectors x and z

$$S(x,z) = \frac{x'z}{x'x + z'z - x'z}$$

A binary variation of this equation which has been widely used in information retrieval, nosology and taxonomy is the so called Tanimoto measure.

Clustering Criteria:

The clustering criterion used may represent a heuristic scheme, or it may be based on the minimization/maximization of a certain performance index.

→ The heuristic approach is guided by intuition and experience.

→ The performance index (objective function) approach is guided by the development of a procedure which will minimize/maximize the chosen performance index. (e.g. sum of squared errors)

$$J = \sum_{j=1}^{N_c} \sum_{x \in S_j} \|x - m_j\|^2 \quad , \quad m_j = \frac{1}{N_j} \sum_{x \in S_j} x$$

where, N_c = no. of clusters

S_j = The set of samples belonging to the j^{th} cluster

m_j = sample mean vector of set S_j

N_j = The no. of samples in S_j

J = sum of the squared errors between the samples of a cluster domain and their mean.

⊙ The K-means clustering algorithm is based on this performance index.

⊙ The Isodata clustering algorithm uses a combination of the heuristic and performance index approaches.

A Simple-Cluster Seeking Algorithm:

Let, a set of N sample patterns $\{x_1, x_2, \dots x_N\}$

First Cluster center $z_1 = x_1$, Threshold $= T \geq 0$

Next, we compute D_{21} from x_2 to z_2, if this distance exceeds T, a new cluster center $z_2 = x_2$ is started.

Otherwise, we assign x_2 to the domain of cluster center z_1. In the next step the distances D_{31} and D_{32} from x_3 to z_1 and z_2 are computed.

If both D_{31} and $D_{32} > T \Rightarrow$ new cluster center $z_3 = x_3$.
Otherwise assign x_3 to the cluster to which it is closest.

The results depend on —

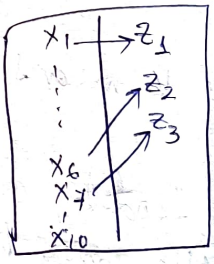
(i) The first cluster center chosen.
(ii) The order in which the patterns are considered.
(iii) The value of T.
(iv) The geometric properties of the data.

Choosing a suitable value of T may be based on —

(i) MST of data points.
(ii) Intra-cluster distance.
(iii) Inter-cluster distance.
(iv) The variance of these cluster domains.
(v) The closest and farthest points in cluster from the center.

Maximin-Distance Algorithm:-

It is a simple heuristic procedure based on the euclidean distance concept.



→ The table is initially empty.
→ We arbitrarily let x_1 become the first cluster center, designated by z_1
→ We determine the farthest sample from x_1 which in this case is x_6 and call it cluster center z_2.
→ We compute distance from each remaining sample to z_1 and z_2, for every pair of these computations we save the minimum distance, then we select max of these minimum distances. If this distance is an appreciable fraction of the distance between cluster centers z_1 and z_2, we call the z_3 corresponding sample cluster center, otherwise the algorithm is terminated.

(v) K-Means Algorithm

Clustering is an unsupervised technique used in discovering inherent structure present in the set of patterns. It aim to extract the groups present in a given dataset and each such group is termed as a cluster.

Let the set of patterns be, $S = \{x_1, x_2, \ldots x_n\} \in \mathbb{R}^m$

where, x_i is the ith pattern vector

n is the total no. of patterns.

m is the dimensionality of the feature space.

① Let, K be the no. of clusters. If the clusters are represented by $C_1, C_2, \ldots C_K$ then we assume —

P1. $C_i \neq \phi$ for $i = 1, 2, \ldots K$

P2. $C_i \cap C_j = \phi$ for $i \neq j$ and

P3. $\bigcup_{i=1}^{K} C_i = S$ (where ϕ = null set)

Clustering → Hierarchical
→ Non-hierarchical / Partitional

② The Partitional clustering problem deals with obtaining an optimal partition of into subsets such that some clustering criterion is satisfied.

→ K-Means algorithm is one of the partitional clustering techniques. (where K needs to be known apriori).

The principal is to minimize the sum of intraclass distances to get the optimal clusters.

② Let, $z_j = \dfrac{\left(\sum_{x \in C_j} x\right)}{\#C_j}$ for $j = 1, 2, \ldots K$

③ Let, $f(C_1, C_2, \ldots C_K) = \sum_{j=1}^{K} \sum_{x \in C_j} \|x - z_j\|^2$ → objective function.

④ Minimize $f(C_1, C_2 \ldots C_K)$ overall such $C_1, C_2, \ldots C_K$ where they satisfy P1, P2 and P3 earlier.

① All possible clusterings of S are to be considered to get the optimal $C_1, C_2 \ldots C_k$. Obtaining exact sol^n is theoritically possible, not feasible in practice.

One requires the evaluation of $S(n,k)$ partitions if exhaustive enumeration is used to solve the problem where —

$$S(n,k) = \frac{1}{k!} \sum_{j=1}^{k} (-1)^{k-j} \binom{k}{j} j^n$$

Thus, approximate heuristic techniques looking for an acceptable sol^n have usually been adopted. One such method is the forgy's k-means algorithm.

Algorithm k-means :-

Step 1 :- Select an initial cluster configuration.

Repeat

Step 2 :- Calculate cluster centers z_j $[j = 1, 2, \ldots k]$ of the existing groups.

Step 3 :- Redistribute pattern among clusters utilizing the minimum squared euclidean distance classifier concept —

$x_i \in C_j$ if $\|x_i - z_j\|^2 < \|x_i - z_l\|^2$

$\forall\, l \in \{1, 2, \ldots k\},\ l \neq j$

Until (There is no change in cluster centers)

End

The behavior of the k-means algorithm is influenced by —

(i) No. of cluster centers specified.
(ii) The choice of initial clusters centers.
(iii) The order in which the samples are taken.
(iv) The geometrical properties of the data.

Merits and Demerits of K-Means Clustering:

- **Merits:**
  - (i) Easy to understand and implement
  - (ii) Efficient, robust and flexible
  - (iii) If data sets are distinct and spherical clusters, then give the best results.

- **Demerits:**
  - (i) Needs prior specification for the no. of cluster centers that is the value of K.
  - (ii) Cannot handle outliers and noisy data, as centroids get deflected.
  - (iii) Does not work well with a very large set of data sets as it takes huge computational time.

Isodata Algorithm :-

The Isodata (Iterative Self-organizing Data analysis) is similar in principle to the K-means procedure in the sense that cluster centres are iteratively determined sample means. Unlike K-means, Isodata represents a fairly comprehensive set of additional heuristic procedures which have been incorporated into an interactive scheme.

A set N_c of initial cluster centers $z_1, z_2, \ldots z_{N_c}$ For a set of N samples, $\{x_1, x_2, \ldots x_N\}$, Isodata consists of the following steps —

Step 1 :- Specify the following process parameters —

K = no. of cluster centers desired.

θ_N = A parameter against which the no. of samples in a cluster domain is compared.

θ_s = standard deviation.

θ_c = lumping parameter

L = max no. of pairs of cluster centers which can be lumped.

I = no. of iterations allowed.

Step 2 :- Distribute the N sample among the present cluster centers using the relation,

$$x \in S_j \quad \text{if} \quad \|x - z_j\| < \|x - z_i\| \qquad i = 1, 2, \ldots N \quad i \neq j$$

S_j represents the subset of samples assigned to cluster center z_j.

Step 3 :- Discard sample subsets with fewer than θ_N members, if for any j, $N_j < \theta_N$, discard S_j and reduce N_c by 1

Step 4 :- Update each cluster center by z_j, setting it equal to the sample mean of its corresponding set S_j,

$$z_j = \frac{1}{N_j} \sum_{x \in S_j} x \quad [\text{where } N_j \text{ is no. of samples in } S_j]$$

Step 5 :- Compute the avg distance D_j of sample in S_j from the corresponding cluster center using the relation

$$D_j = \frac{1}{N_j} \sum_{x \in S_j} \|x - z_j\| \quad , j = 1, 2, \cdots N_c$$

Step 6 :- Compute the overall avg distance of the samples from their respective cluster center, using

$$D = \frac{1}{N} \sum_{j=1}^{N_c} N_j D_j$$

Step 7 :- (a) If this is the last iteration, set $\theta_c = 0$ go to step 11

(b) If $N_c \leq K/2$ go to step 8

(c) If even-numbered iteration or if $N_c \geq 2K$ go to step 11, otherwise continue.

Step 8 :- Find standard deviation Vector

$$\sigma_j = (\sigma_{1j}, \sigma_{2j} \cdots \sigma_{nj}) \text{ for each sample subset, using}$$

$$\sigma_{ij} = \sqrt{\frac{1}{N} \sum_{x \in S_j} (x_{ix} - z_{ij})^2} \quad , i = 1, 2, \cdots n \; , j = 1, 2, \cdots N_c$$

where n is sample dimensionality.

x_{ix} is the ith component of kth sample

z_{ij} is the ith " " z_j

σ_j is the standard deviation of the samples in S_j along a principal coordinate axis.

Step 9 :- find the max component of each σ_j and denote it by $\sigma_{j\max}$

Step 10 :- If for any $\sigma_{j\max}$ $[j = 1, 2, \ldots N_c]$ we have

$\sigma_{j\max} > \theta_s$ and

(a) $D_j > \bar{D}$ and $N_j > 2(\theta_N + 1)$ or

(b) $N_c \leq K/2$

then split z_j into two new cluster centers z_j^+ and z_j^- delete z_j and increase N_c by 1

→ z_j^+ is formed by adding a given quantity γ_j to the component of z_j

→ z_j^- is formed by subtracting γ_j from the same component of z_j.

$\gamma_j = K \sigma_{j\max}$ where $0 < K \leq 1$

→ γ_j should be sufficient to provide a detectable difference in the distance from an arbitrary sample to the two new cluster centers, but not so large to change the overall cluster domain arrangement.

If splitting took place here, goto Step 2 otherwise continue.

Step 11 :- Compute the pairwise distances D_{ij} between all cluster centers —

$$D_{ij} = \|z_i - z_j\| \quad i = 1, 2, \ldots N_c - 1, \quad j = i+1, \ldots N_c$$

Step 12 :- Combine the distances D_{ij} again θ_c, Arrange the L smallest distances which are less than θ_c in descending order

$[D_{i_1 j_1}, D_{i_2 j_2}, \ldots D_{i_L j_L}]$ L = max no. of pairs of cluster centers can be lumped together.

where $D_{i_1 j_1} < D_{i_2 j_2} < \ldots < D_{i_L j_L}$

Step 13 :- With each distance, $D_{i,jl}$ → associated pair of cluster centers z_{il}, z_{jl}

For $l = 1, 2, \ldots L$, if neither z_{il} nor z_{jl} has been used in lumping in this iteration, merge these two cluster centers using the relation —

$$z_l^* = \frac{1}{N_{il} + N_{jl}} \left[N_{il}(z_{il}) + N_{jl}(z_{jl}) \right]$$

Delete z_{il} and z_{jl} and reduce N_c by 1

Step 14 :- If this is the last iteration, the algorithm terminates

Otherwise goto step 1 if any of the parameters requires changing at the user's discretion or go to step 2 if the parameters are to remain same from the next iteration

Evaluation of clustering Results:

The principal difficulty in evaluating the results of clustering algorithms is inability to visualize the geometrical properties of a high-dimensional space.

A very useful interpretation tool is the distance between cluster centers. This information is best presented in the form of a table.

If two cluster centers are relatively close and one of the centers is associated with a much larger no. of samples, it is often possible to merge the two cluster domains. The variances of a cluster domain about its mean can be used to infer the relative distribution of the samples in the domain.

If some clusters centre is far from others, if it is associated with numerous samples, we would accept it as a valid description of data, otherwise we might dismiss the center as representative of noise samples.

If some cluster domain has very similar variances, it can be expected to be roughly spherical in nature.

If Cluster domain is significantly elongated about the 3rd coordinate axis.

This information, coupled with the distance table and sample numbers can be of a significant value in interpreting clustering results.

Graph-Theoretic Approach:

An alternative approach to cluster seeking is to make use of some basic notion in graph theory. In this approach, a pattern graph is first constructed from the given sample patterns, which form the nodes of the graph. A node j is connected to a node k by an edge, if the patterns corresponding to these two nodes are similar or related.

Pattern X_j and Pattern X_k are said to be similar if the similarity measure $S(X_j, X_k)$ is greater than a prespecified threshold 'T'. It may be used to generate a similarity matrix S, whose elements are 0 or 1. The matrix provides a systematic way to construct the pattern graph. Since the cliques of a pattern graph form the clusters of the patterns, cluster seeking can be accomplished by detecting the cliques of the pattern graph. Several clique detection algorithms are there, Several such methods exist that utilize the MST of the data points to get the clustering.

Bayesian Classification :-

- A statistical classifier which performs probabilistic prediction i.e. predicts class membership probabilities.
- Performance :- (i) It can solve problems involving both categorical and continuous valued attributes.
  (ii) A simple Bayesian classifier, naive bayesian classifier has comparable performance with that of decision tree and ANNs.

- Let 'X' be a data sample, H be a hypothesis that X belongs to class C.

Classification is to determine $P(H/X)$, the probability that the hypothesis holds given the observed data sample X.

$P(H)$ [prior probability] :- The initial probability

$P(X)$ [Evidence] → Probability that sample data is observed.

$P(X|H)$ → (Likelihood) → The Probability of observing the sample X given that the hypothesis holds.

By Baye's Theorem,

$$P(H|X) = \frac{P(X|H)\,P(H)}{P(X)}$$

Naive Bayes Classifier

Now,

Let D be a training set of tuples with their associated class labels, and each tuple is represented by an n attribute vector $X = (x_1, x_2, \ldots x_n)$, There are K classes, $C_1, C_2, \ldots C_K$

$$P(C_K|X) = \frac{P(C_K, X)}{P(X)} = \frac{P(C_K)\,P(X|C_K)}{P(X)}$$

The denominator is const. as all the values of the features are given.

The numerator is equivalent to the joint probability model $P(C_k, x_1, \dots x_n)$

$$P(C_k)\ P(X|C_k) = P(C_k, X)$$

where, $X = (x_1, x_2, \dots x_i, \dots x_n)$

Using the chain rule for repeated applications of definitions of conditional probability we can write —

$$P(C_k, x_1, \dots x_i, \dots x_n) = P(x_1, x_2, \dots x_n, C_k)$$

$$= P(x_1|x_2, \dots, x_i, \dots x_n, C_k)\ P(x_2, \dots x_i, \dots x_n, C_k)$$

$$= P(x_1|x_2, \dots x_i, \dots x_n, C_k)\ P(x_2|x_3, x_i, \dots x_n, C_k)$$

$$P(x_3, \dots x_i, \dots x_n, C_k)$$

$$\dots$$

$$= P(x_1|x_2, \dots x_i, \dots x_n, C_k) \dots P(x_{i-1}|x_i, \dots x_n, C_k)$$

$$\dots P(x_{n-1}|x_n, C_k)\ P(x_n|C_k)\ P(C_k)$$

"Naive" conditional independence ⇒ assuming that each feature x_i is conditionally independent of every other feature x_j for $j \neq i$, given the category C_k,

$$P(x_i|x_{i+1}, \dots x_n, C_k) = P(x_i|C_k)$$

Thus, the joint model can be expressed as —

$$P(C_k|x_1, \dots x_n) \propto P(C_k, x_1, \dots x_n)$$

$$= P(C_k)\ P(x_1|C_k)\ P(x_2|C_k) \dots P(x_n|C_k)$$

$$= P(C_k) \prod_{i=1}^{n} P(x_i|C_k)$$

The naive Bayes classifier combines this model with a decision rule.

① One common rule is to pick the hypothesis that is most probable, this is known as the maximum a posteriori or MAP decision rule.

② The corresponding classifier, a Bayes classifier is the function that assigns a class label $\hat{y} = C_k$ for some k as follows —

$$\hat{y} = \underset{k \in \{1,\dots K\}}{\operatorname{argmax}} \; P(C_k) \prod_{i=1}^{n} P(x_i | C_k)$$

(i) If A_i is categorical $P(x_i | C_k)$ is the no. of tuples in C_k having value x_i for A_i divided by $|C_k, D|$ (# no of tuples of C_k in D)

(ii) If A_i is continuous-valued $P(x_i | C_k)$ is usually computed based on Gaussian distribution with a mean μ and standard deviation σ.

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$\text{So, } P(x_i | C_k) = g(x_i, \mu_{C_k}, \sigma_{C_k})$$

→ Avoiding the 0-probability problem → Naive Bayesian prediction requires each conditional probability be non-zero, otherwise the posterior probability will be zero.

$$P(X | C_i) = \prod_{k=1}^{n} P(x_k | C_i)$$

So, use Laplacian correction or estimator [Ex:- adding 1 to each case]

Advantages:-

(i) Easy to implement

(ii) Good results obtained in most of the cases.

Disadvantages:-

(i) Assumption:- class conditional independence, Not always valid for real life problems, since dependencies do exist among variables (like hospitals, patients name, age etc)

(ii) Dependencies among these cannot be modeled by Naive Bayesian classifier.

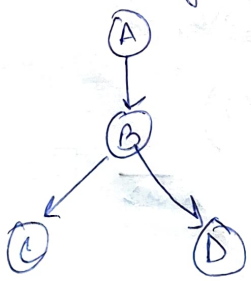
Advantage of Naive Bayes over Decision Tree:-

(i) Decision tree has a tendency to be over fitted with the training data.

(ii) Decision tree are not generally suitable for application like diagnosis of Cancer, As Cancer does not occur in the population in large numbers, it may get pruned out more likely by decision tree.

Bayesian Network:-

A Bayesian network is a graphical model for depicting probabilistic relationships among a set of variables. It's represented in the form of DAG. It is called also as a Bayes Network, Belief Network, Decision Network, Bayesian Model.



DAG → uses conditional probability for independent nodes and each of the dependent nodes is associated with a Conditional probability table (CPT)

(i) Bayesian network encodes the conditional indepence relationships between the variables in the graph structure.

(ii) Provides a compact representation of the joint probability distribution among the variables.

(iii) A problem domain is modeled by a list of variables

$X_1, \ldots X_n$

(iv) Knowledge about the problem domain is represented by a joint probability $P(X_1, \ldots X_n)$

(v) Directed links represents casual direct influences.

(vi) Each dependent node has a conditional probability table quantifying the effects from the parents.

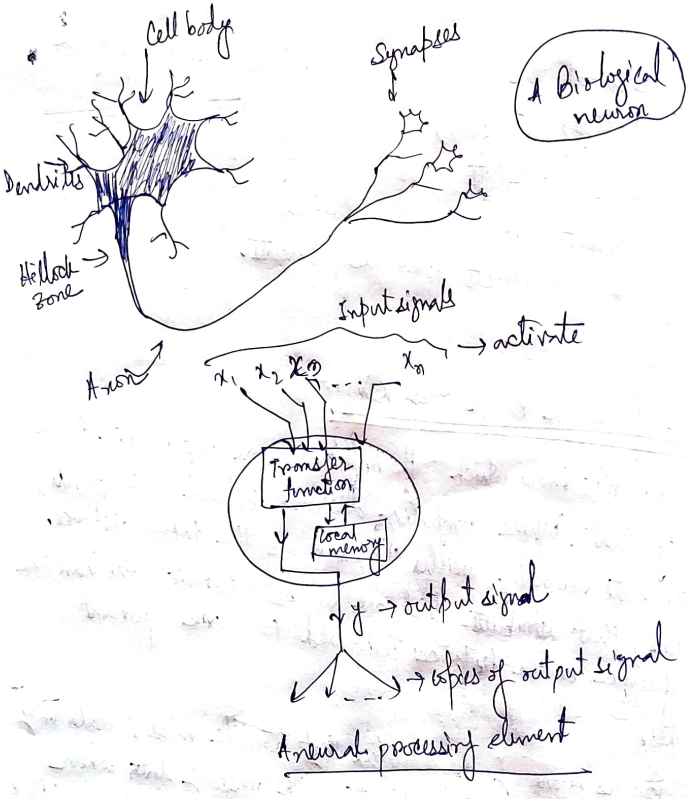
(vii) No directed cycles.

Advantage of Naive Bayes over KNN:-

(i) KNN doesn't know which attributes are more important. As a result of that, while computing distance between data points, each attribute normally weighs the same to the total distance. This means, that attributes which are not so important will have the same influence on the distance compared to more important attributes.

(ii) Naive Bayes is one of the classifiers that handle missing data very well, it just excludes the attribute with missing data when computing posterior probability. With KNN, one can't do classification if there is any missing data. The reason is that, distance is undefined if one or more of attributes are missing, unless these attributes are being omitted while computing distance. As a consequence, we need to rely on common solutions for missing data, e.g. imputing avg values.

iii) KNN needs one parameter more than Naive Bayes. This is the no. of neighbours (k). This means one need to do model selection for KNN in order to determine the best values of K.



Neural Network:

An artificial Neural network is a parallel distributed information processing structure in the form of a directed graph, with —

(a) The nodes of the graph are called processing elements.

(b) The links of the graph are called connections. Each connection function as an instantaneous unidirectional signal-conditioning path.

(c) Each processing element can receive any no. of incoming connections (also called input connections).

(d) Each processing element can have any no. of outgoing connections, but the signals in all of these must be the same.

(e) Processing elements can have local memory.

(f) Each processing element possesses a transfer function, which can use (and alter) local memory, can use input signals and which produces the processing element's output signal.

Transfer functions can operate continuously or episodically.

→ If they operate episodically, there must be an input called 'activate' that causes the function to operate on the current input signals and local memory values and to produce and update output signal.

→ Continuous processing elements are always operating. The "activate" input arrives via a connection from a scheduling processing element that is part of the network.

(g) Input signals to a neural network from outside the network arrive via connection that originate in the outside world. Outputs from the network to the outside world are connections that leave the network.

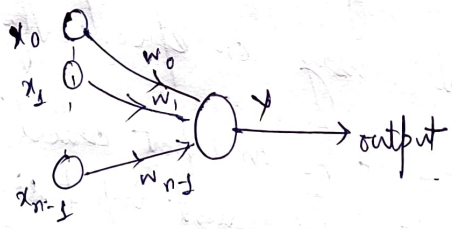
The general artificial neuron model has 5 components —

1. A set of inputs, X_i
2. A set of weights, W_i
3. A bias, θ
4. An activate function, f
5. Neuron output, Y

Single Layer Perceptron:

The single layer perceptron consists with only one node that can be used with both continuous valued and binary inputs. A perceptron that decides whether an input belongs to one of two classes.

The single node computes the weighted sum of the input elements subtracts a threshold (θ) and passes the result through a hard limiting nonlinearity such that the output y is either +1 or -1 for class A and class B respectively.



$$Y = f\left(\sum_{i=0}^{n-1} w_i x_i - \theta\right)$$

$$y = \begin{cases} +1 \rightarrow \text{class A} \\ -1 \rightarrow \text{class B} \end{cases}$$

The perceptron forms two decision regions separated by a hyperplane which in 2-D is a line. The equation of the boundary line depends on the connection weights and the threshold. Connection weights and the threshold can be fixed or adapted using a no. of different algorithms.

The Perceptron Convergence Procedure:

Step-① Initialize weights and Threshold:

Set $w_i(t)$: $(0 \le i \le N-1)$ and θ to small random values.

$w_i(t)$ → weight from input i at time t

θ → Threshold in the output node.

Step-② Present New input and Desired output;

Present new continuous valued input $x_0, x_1, \ldots x_{N-1}$ along with the desired output $d(t)$.

Step-③ Calculate actual output;

$$y(t) = f\left(\sum_{i=0}^{N-1} w_i(t) x_i(t) - \theta\right)$$

Step-④ Adapt Weights:

$$w_i(t+1) = w_i(t) + \eta \left[d(t) - y(t)\right] x_i(t) \qquad 0 \le i \le N-1$$

$$d(t) = \begin{cases} +1 & \text{if inputs from class A} \\ -1 & \text{if inputs from class B} \end{cases}$$

η → positive gain fraction less than 1.

$d(t)$ → desired correct output.

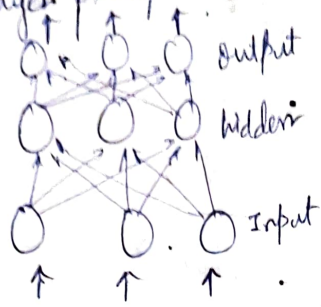
Step-⑤ Repeat by Going to Step-2;

The gain term η must be adjusted to satisfy the conflicting requirements of fast adaptation for real changes in the input distributions and averaging of past inputs to provide stable weight estimates.

Problem :- Decision boundaries may oscillate continuously when inputs are not separable and distributions overlap.

Multi-layer Perceptron:

Multi layer perceptrons are feed-forward nets with one or more layers (called hidden layers) of nodes between the input and output nodes. A 3-layer perceptron with 2 layers of hidden units is-



The decision boundary provided by SLP

$$y(t) = f\left(\sum_{i=0}^{N-1} w_i(t)\, x_i(t) - \theta\right)$$

Letting $\theta = 0$, we get,

$$y(t) \begin{cases} > 0 & \text{if input from class } w_1 \\ < 0 & \text{if input from class } w_2 \end{cases}$$

If we want to find a solution weight vector W with the property that $W'X > 0$ for all patterns of w_1 and $W'X < 0$ for all patterns of w_2.

If patterns of w_2 are multiplied by -1, we obtain the equivalent condition $W'X > 0$ for all patterns.

$N \rightarrow$ total no. of augmented sample patterns in both classes.

Now, finding a vector W such that the system of inequalities $W'X > 0$ is satisfied. $X = \begin{bmatrix} x_1' \\ x_2' \\ \vdots \\ x_N' \end{bmatrix}$

$W = (w_1, w_2, \cdots w_n, w_{n+1})'$ and 0 is the zero vector.

If there exists a W then it is consistent otherwise inconsistent.

If $X(t) \in w_1$ and $W'(t)X(t) > 0$ let $W(t+1) = W(t)$ otherwise replace $W(t)$ by, $W(t+1) = W(t) + \eta X(t)$ [η = correction factor]

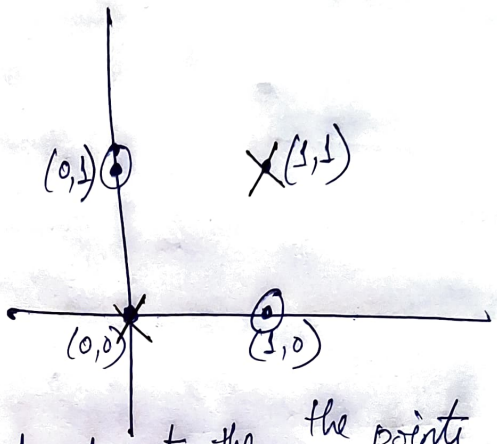
If $X(t) \in W_2$ and $W'(t)X(t) < 0$ let $W(t+1) = W(t)$
otherwise replace $W(t)$ by $W(t+1) = W(t) - \eta X(t)$

Can single layer perceptron learn an XOR function? Justify

Single layered perceptron is capable of representing only linear separable data.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Truth Table of XOR



By using any means we cannot separate the the points with value 1 and value 0 with the help of a single line

SYNTACTIC PATTERN RECOGNITION

(a) Statistical pattern recognition attempts to classify patterns based on a set of extracted features and an underlying statistical model for the generation of these patterns. Ideally, this is achieved with a rather straightforward method.

→ determine the feature vector
→ train the system
→ classify the patterns.

Patterns that include structural or relational information are difficult to represent as feature vectors. Syntactical PR uses this structural information for classification and description.

→ Grammars can be used to create a definition of the structure of each pattern class.

→ Then, recognition and classification are done using either —

(a) Parsing using formal grammars (called SPR)

OR

(b) Relational graph matching (called Hierarchical approach)

Classification :-

Classification can be done based on a measure of structural similarity in patterns. Each pattern class can be represented by a structural representation or description.

(a) Description:-

A description of the pattern structure is useful for recognizing entities when a simple classification isn't feasible. It can also describe aspects that cause a pattern not to be assigned to a particular class.

In complex cases recognition can only be achieved through a description for each pattern rather than through classification.

→ The simplest sub-patterns are called pattern primitives and should be much easier to recognize than the overall patterns.

→ The language used to describe the structure of the patterns in terms of sets of pattern primitives is called the pattern description language. The pattern description language will have a grammar that specifies how primitives can be composed into patterns.

Syntax Analysis

When a primitive within a pattern is identified, syntax analysis (parsing) is performed on the sentence describing the pattern to determine if it is correct w.r.t. the grammar. It also gives a structural description of the sentence associated with the pattern.

Advantage → A grammar (rewriting) rule can be applied many times.

Pattern description using Grammar:-

A grammar describing 4 blocks arranged in 2 block stacks

$V_T = \{table, block, +, \uparrow\}$ (terminal symbols)

$V_N = \{DESK, LEFT\ STACK, RIGHT\ STACK\}$ (non-terminal symbols)

$S = DESK \in V_N$ (root symbol)

$P = \{$ DESK → LEFT STACK + RIGHT STACK

DESK → RIGHT STACK + LEFT STACK

LEFT STACK → block ↑ block ↑ table

RIGHT STACK → block ↑ block ↑ table $\}$

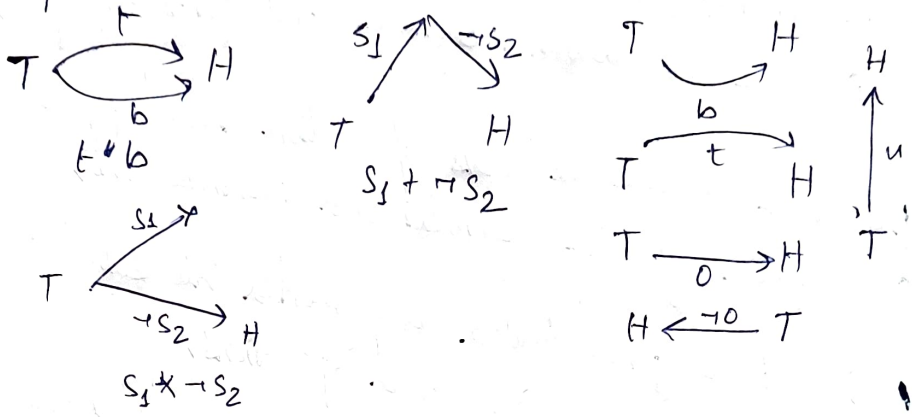
(Production rules)

A set of terminal symbols $\{t, b, u, 0, s, *, -, +\}$ where

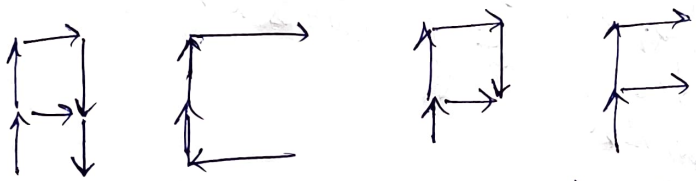
'+' represents head to tail concatenation

'*' represents head-head and tail-tail attachment

'−' represents the head-tail reversal



(a)



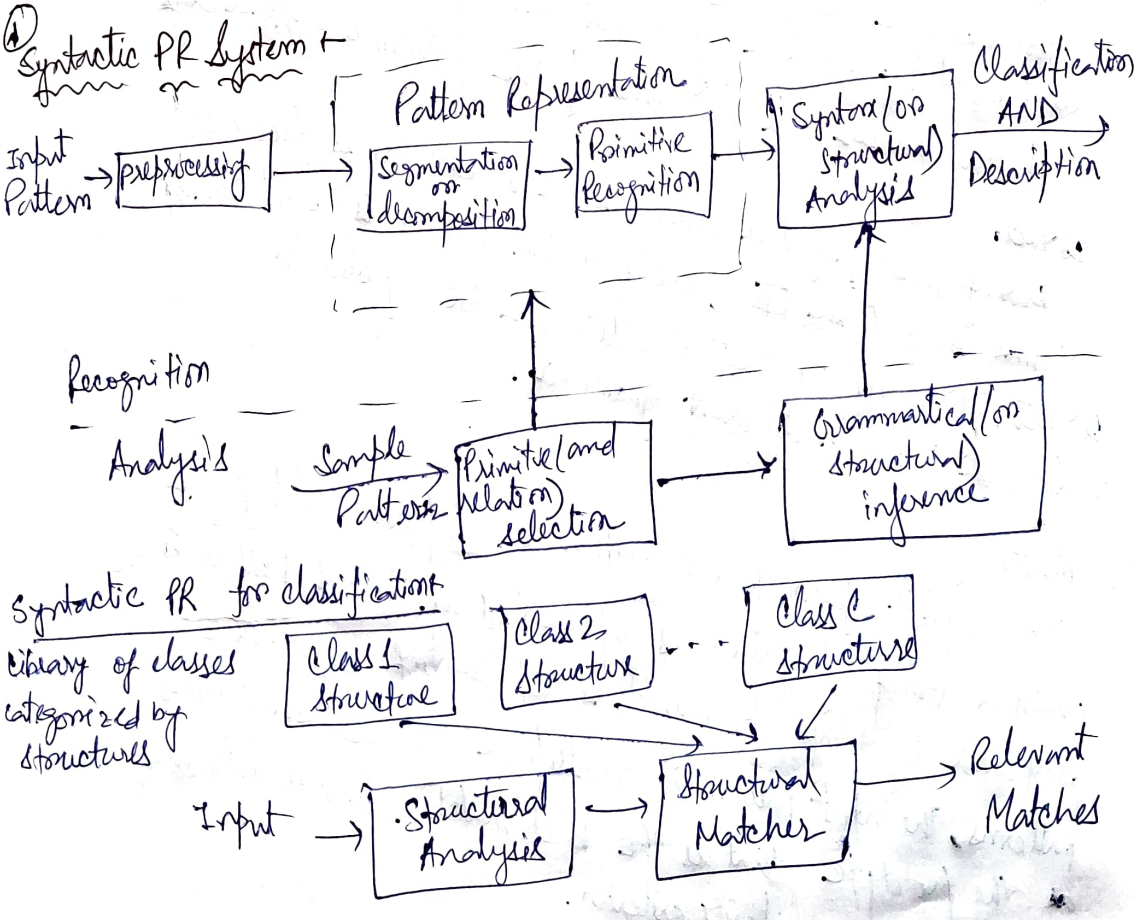
$A = u + \left((u + 0 + \neg v) \cdot 0\right) + \neg v$

$P = u + \left((u + 0 + \neg v) \cdot 0\right)$

$F = ut + (0 \times v) + 0$

$C = \neg 0 + u + u + 0$

(b) Syntactic PR System



Syntactic PR-system consists of two main parts —

(i) Analysis → primitive selection and grammatical / structural inference

(ii) Recognition → Preprocessing, segmentation or decomposition, primitive and relation recognition and syntax analysis.

→ Preprocessing includes the tasks of pattern encoding and approximation, filtering, restoration and enhancement,

→ After preprocessing, the pattern is segmented into sub-patterns and primitives using predefined operations.

→ Sub-patterns are identified with a given set of primitives, so each pattern is represented by a set of primitives with the specified syntactic operations.

In Syntax Parsing,

for example, using the concatenation operation, each pattern is recognized by a string of concatenated primitives. At this point, the parser will determine if the pattern is syntactically correct.

During parsing/syntax analysis, a description is produced in terms of a parse tree, assuming the pattern is syntactically correct. If it isn't correct it will either be rejected or analyzed based on different grammar.

In Matching,

The simplest form of recognition is template matching, in which a string of primitives representing an input pattern is compared to strings of primitives representing reference patterns. The input pattern is classified in the same class as the prototype that is the best match, which is determined by a similarity criterion.

⑥ Template Matching vs Complete Passing :-

Matching	Passing
① The structural description is ignored.	① Complete passing uses the entire structural description.
Efficiency of the recognition process is improved by simpler approaches that do not require a complete passing.	Passing is required if the problem necessitates using a complete pattern description for recognition.
	Passing can be expensive, so don't use it unnecessarily.

There are many intermediate approaches, for example, a series of tests designed to test the occurrence of certain primitives, and pattern or combinations of these. The result of these tests will determine a classification.

Inferring Grammars :-

Grammatical inference machine similar to "learning" in the discriminant approach, it infers a grammar from a set of training patterns. The inferred grammar can then be used for pattern description and syntax analysis.

Hierarchical Approach :-

It comes from the similarity that can be seen between the structure of patterns and the syntax or grammar of languages.

Basic idea:- Use graphs to represent structural relationships

→ Primitives : nodes / vertices of a graph

→ Relationships : arcs / edges of a graph

essential component : graph matching.

Homomorphism → (a) A function that relabels one graph into another.
(b) Merging nodes is allowed so long as it preserves the edge relationships.

Isomorphism → (a) A homomorphism that is 1-to-1 and onto
(b) Merging nodes not allowed - exact match with nothing changed but labels.

Quick rejection criteria:-

(•) For a match to exist, several things must hold →
- → Same no. of vertices
- → Same no. of nodes
- → In-degree of vertices must match
- → Out-degree of vertices must match
- → Cycles and lengths must match

(•) Failure of any of these to match means you can reject the match, but determine the isomorphism requires that you actually find the relabeling.

Finding Isomorphisms →
- → Use an adjacency matrix representation of the graph
- → Reorder the rows and corresponding columns
- → Compare all possible reorderings
- → Comparison: $O(n^2)$
- → Possible permutations: $O(n!)$ (only feasible for small n)

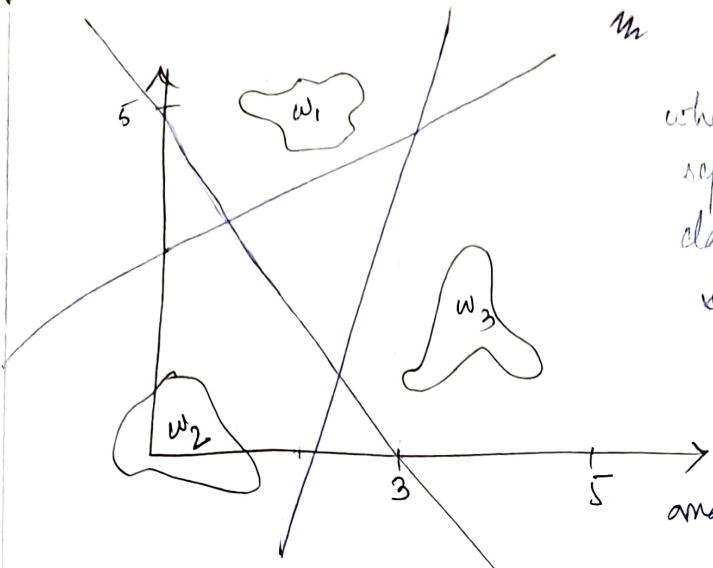
Subisomorphism → (Is one pattern part of another?)
Equivalent to finding subgraph that is isomorphic to another → subisomorphism.

Finding subisomorphisms not only requires determining isomorphism between graph and subgraph, but also considering all possible subgraphs.

Graph Similarity

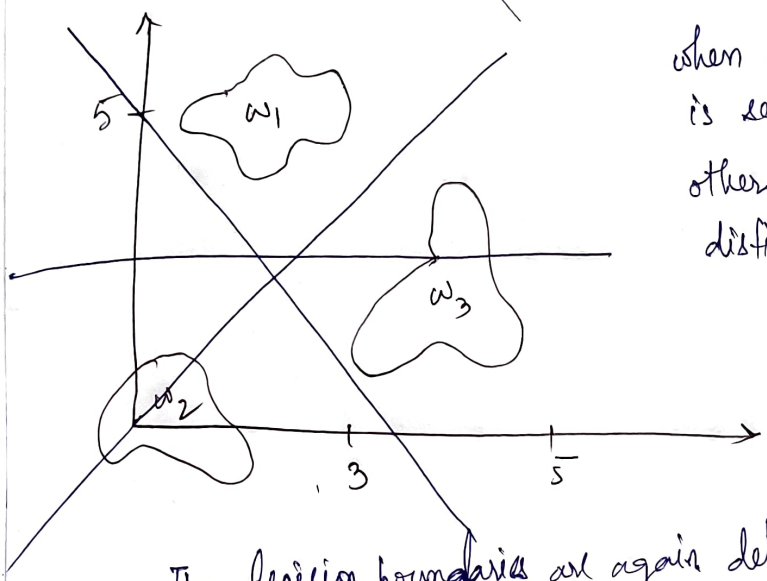
To count modifications required to make one match to the other

- → inserting an edge
- → deleting " "
- → inserting a node
- → deleting " "
- → splitting " "
- → merging two nodes



when each class is separable from the other classes by a single decision surface.

If x belongs to class w_1 then $d_1(x) > 0$ and $d_2(x) < 0$, $d_3(x) < 0$



when each pattern class is separable from every other individual class by a distinct decision surface.

The decision boundaries are again determined by setting the decision function equal to zero. But now, if x belongs to class w_1 then $d_{12}(x) > 0$ and $d_{13}(x) > 0$

Supervised PR

(i) Uses known and labeled data as input.

(ii) The no. of classes is known.

(iii) Less Computational complexity

Unsupervised PR

(i) Use unlabeled data as input

(ii) No. of clusters may or may not be known.

(iii) More computational complexity